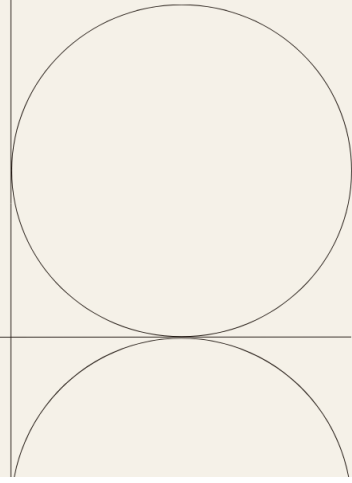


Project Documentation



1. Overview

<i>Problem Name</i>	Missing Integer
<i>Problem Context</i>	Given an array A of N integers, find the smallest positive integer (greater than 0) that does not occur in A. • Example 1: A = [1, 3, 6, 4, 1, 2] → Returns 5 • Example 2: A = [1, 2, 3] → Returns 4 • Example 3: A = [-1, -3] → Returns 1
<i>Constraints</i>	The solution is designed to meet strict performance requirements: • Input Size: Up to 100,000 integers. • Value Range: Integers between -1,000,000 and 1,000,000.
<i>Project Members</i>	Yousef.A, Yousef.M, Razan, Rahma, Dina, Lojain

Objectives

- **Compare Approaches:** Develop and evaluate two different solutions: a **Recursive** method and an **Iterative (Cyclic Sort)** method.
- **Optimize Performance:** Ensure the code runs in **Linear Time** to handle large datasets of up to **100,000** elements efficiently.
- **Manage Resources:** Minimize memory usage.
- **Handle Edge Cases:** Guarantee the algorithm works correctly for negative numbers, zeros, and perfectly sorted sequences.



1. Team Roles

Task	Assignee
Write the actual code for the non-recursive (iterative) approach.	@Razan Abdelkhaleq
Write the step-by-step Pseudo code for Member 1's algorithm, and Perform Complexity Analysis section.	@Rahma Essam

Write the actual code for the recursive approach.	@Yousef El-Ansari
Write the step-by-step Pseudo code for Member 3's algorithm, and Perform Complexity Analysis section.	@Yousef Michael
prepare the comparison between the two algorithms in terms of performance, complexity, and test cases, and write the final document combining all project work.	@Dina Ahmed @Lojain Hussien



1. Pseudocode

3.1 Recursive:

```
recursive pseudocode

1 algorithm find_miss( cur, pres, max_val )
2 {
3   if (cur > max_val OR NOT pres[cur]) then
4     return cur
5   return find_miss(cur + 1, pres, max_val)
6 }
7
8 algorithm solve()
9 {
10  write "Enter the number of elements (N): "
11  read n
12  if reading fails then return
13  if n < 0 then n ← 0
14  pres ← array of size n + 2 filled with FALSE
15  write "Enter the elements: "
16  for i ← 0 to n - 1 do
17  {
18    read x
19    if (x > 0 AND x ≤ n) then
20      pres[x] ← TRUE
21  }
22  result ← find_miss(1, pres, n + 1)
23  write "The smallest missing positive integer is: " , result
24 }
25
26 Main ()
27 {
28   solve()
29 }
```

3.2 Non-Recursive:

```
iterative pseudocode

1  algorithm MissPos( A, n )
2  // A: input array, n: size of the array
3  {
4      for i ← 0 to n-1 do
5      {
6          while A[i] > 0 AND A[i] ≤ n AND A[i] ≠ A[A[i]-1] do
7          {
8              temp ← A[i]
9              A[i] ← A[temp-1]
10             A[temp-1] ← temp
11         }
12     }
13
14     for j ← 0 to n-1 do
15     {
16         if A[j] ≠ j+1 then
17             return j+1
18     }
19
20     return n + 1
21 }
```



1. Code

4.1 Recursive:

```
1 #include <bits/stdc++.h> // المكتبة التي فيها كل الداء
2 using namespace std;
3
4 #define ll long long
5 #define forn(i, n) for (int i = 0; i < int(n); i++) // اختصار عشان متفقدش نكتب كبير ونكتب صوابا
6 #define el "\n"
7
8 // فلكتش الريبكشن دي بغي التي بفصل تدور ورا الرقم لحد ما نجيبه من فاه
9 // استخدمنا الـ (&) عشان الميموري غالبة علينا ومش علزبن نيمزفها ونأخذ منها نسخ كبير
10 int find_miss(int cur, vector<bool> &pres, int max_val) {
11
12     // لو وصلنا للآخر خالص أو لينا مكان "فلمنى" في الخريطة ببقى هو ده الرقم التلي با معلم
13     if (cur > max_val || !pres[cur]) {
14         return cur;
15     }
16
17     // لو لسه ملغفلش، بتنادي المالكشن تاني ونقولها: "شوفي التي بعده يا ست الكل"
18     return find_miss(cur + 1, pres, max_val);
19 }
20
21 void solve() {
22     int n;
23
24     // بتطلب من اليورن بدخل عدد الحليس بكل أدب وشركة
25     cout << "Enter the number of elements (N): " ;
26     if (!cin >> n) return;
27
28     // لو اليورن حب يهزر ودخل رقم سالب، هختبره صفر ونمشي حاليًا
29     if (n < 0) n = 0;
30
31     // كلها كذب لحد ما نلافي الرقم ونخليها صح bool بنجيب
32     vector<bool> pres(n + 2, false);
33
34     if (n > 0) {
35         cout << "Enter the " << n << " element/s separated by space: " ;
36         forn(i, n) {
37             int x;
38             cin >> x;
39
40             // بفعلس الأرقام فا الموجب بس والتي "ع الحركوك" جوه نطلق الأراي هو التي بهما
41             if (x > 0 && x <= n) {
42                 pres[x] = true; // علم عليه يا بلشا إنه جه وشرفنا
43             }
44         }
45     }
46
47     // اللحظة الحاسمة .. بتطلع النتيجة النهائية لليورن
48     cout << "The smallest missing positive integer is: " << find_miss(1, pres, n + 1) << el;
49 }
50
51 int main() {
52     // عشان المدخلات والمخرجك تجري طيلة C القربو بناف الكود .. بفعل الربط مع
53     // عشان كل واحد بفعل في سكة لوحده بدون ما يستنى التالي cout عن cin بفصل
54     //ios_base::sync_with_stdio(false);
55     //cin.tie(NULL);
56
57     solve(); // نوكلنا على الله
58
59     return 0; // ثم بحمد الله والبرهانج زي التل
60 }
```

4.2 Non-Recursive:

```
iterativeMissingInt.cpp

1 #include <bits/stdc++.h> /// الشنطة اللي فيها كل الحدة والآلات الحادة
2 using namespace std;
3
4 /// الفالكشن دي بفي اللي مختص المصلحة "من غير ريكورجن" ومن غير وجع دماغ
5 /// (O(1) Space) صفا ال & هنا عشان ندخل في الاري الاصلية وياوفر ميموري
6 int miss_pos(vector<int> &A) {
7     int n = A.size();
8
9
10    /// أول لفه: بنقرز الأرقام وكل واحد بروح بقدر في بينهم
11    for (int i = 0; i < n; i++) {
12        /// طول ما الرقم موجب و عطي الحركوك جوه المصفوفة وممن في مكانه .. بدله با ممل
13        while (A[i] > 0 && A[i] ≤ n && A[i] ≠ A[A[i] - 1]) {
14            int temp = A[i]; /// بتركز الرقم ده على جنب ثانية
15            A[i] = A[temp - 1]; /// بنجيب اللي في مكانه الصح هنا
16            A[temp - 1] = temp; /// ونبخته هو لبيته اللي بسايله
17        }
18    }
19
20    /// ثاني لفه: بنفرض المصفوفة بيت بيت ونشوف مين اللي لسه "غائب"
21    for (int i = 0; i < n; i++) {
22        if (A[i] ≠ i + 1) {
23            return i + 1; /// فقتنا الرقم الثاني .. يرجع فوراً
24        }
25    }
26
27    /// لو كله تمام، يبقى الرقم اللي عليه الدور هو اللي ناقص
28    return n + 1;
29 }
30
31 int main() {
32    /// شيلنا التريو عشان الكسك يظهر لليوزر في وفه الصح
33    //ios_base::sync_with_stdio(false);
34    //cin.tie(NULL);
35    int n;
36    cout << "Enter the number of elements (N): " ;
37    if (!(cin >> n)) return 0;
38
39    if (n ≤ 0) {
40        cout << "The smallest missing positive integer is: 1" << endl;
41        return 0;
42    }
43
44    vector<int> A(n);
45    cout << "Enter the " << n << " element/s separated by space: " ;
46    for (int i = 0; i < n; i++) {
47        cin >> A[i];
48    }
49
50    cout << "The smallest missing positive integer is: " ;
51    cout << miss_pos(A) << endl;
52
53    return 0; /// ثم بصد الله وكله زي الفل
54 }
55
```



1. Testing

5.1 Recursive:

<i>Testing Logic</i>	<i>Input</i>	<i>Expected Output</i>	<i>Output</i>
boundary n+1	[1 ,2 ,3]	4	4
General gap detection	[1, 3, 6, 4, 1, 2]	5	5
Ignoring non- positives	[-1 ,-3]	1	1
Handling repetition	[1 , 1 , 2 , 4]	3	3

Single Element (Best Case for search)	[1]	2	2
Empty Input	[]	1	1
Zero exclusion	[0 , 2 , 3]	1	1
Out-of- bounds values	[1 , 500 , 2]	3	3

5.2 Non-Recursive:

<i>Testing Logic</i>	<i>Input</i>	<i>Expected Output</i>	<i>Output</i>
boundary n+1	[1 ,2 ,3]	4	4
General gap detection	[1, 3, 6, 4, 1, 2]	5	5
Ignoring non- positives	[-1 ,-3]	1	1
Handling repetition	[1 , 1 , 2 , 4]	3	3
Single Element (Best Case for search)	[1]	2	2
Empty Input	[]	1	1

Zero exclusion	[0 , 2 , 3]	1	1
Out-of- bounds values	[1 , 500 , 2]	3	3



5.3 Authentic Screenshots

```
Enter the number of elements (N): 3
Enter the 3 element/s separated by space: 1 2 3
The smallest missing positive integer is: 4
```

```
Enter the number of elements (N): 6
Enter the 6 element/s separated by space: 1 3 6 4 1 2
The smallest missing positive integer is: 5
```

```
Enter the number of elements (N): 2
Enter the 2 element/s separated by space: -1 -3
The smallest missing positive integer is: 1
```

```
Enter the number of elements (N): 4
Enter the 4 element/s separated by space: 1 1 2 4
The smallest missing positive integer is: 3
```

```
Enter the number of elements (N): 1
Enter the 1 element/s separated by space: 1
The smallest missing positive integer is: 2
```

```
Enter the number of elements (N): 0
The smallest missing positive integer is: 1
```

```
Enter the number of elements (N): 3
Enter the 3 element/s separated by space: 0 2 3
The smallest missing positive integer is: 1
```

```
Enter the number of elements (N): 3
Enter the 3 element/s separated by space: 1 500 2
The smallest missing positive integer is: 3
```



1. Complexity

6.1 Recursive:

6.1.1 Time : $O(n)$

- Solve 1 (Logic & Basic Operations):

We calculate time complexity by counting how many basic operations the computer performs based on the input size N .

Step 1 (Initializing the array):

```
vector<bool> pres(n + 2, false);
```

To create a vector of size $N+2$ and set all values to false, the computer visits every single slot.

Time taken: $O(N)$

Step 2 (Reading and filtering the input):

The `for(n(i, n))` loop runs exactly N times. Inside the loop, checking `x > 0 && x <= n` and assigning `pres[x] = true` are instant, basic math operations. Since it does a basic operation N times, the time scales linearly with the input.

Time taken: $O(N)$

Step 3 (The Recursive Search):

```
find_miss(1, pres, n + 1)
```

In the **Worst-Case Scenario** (for example, if your array is $A = [1, 2, 3, 4, 5]$), the function makes exactly $N+1$ recursive calls to find the missing integer.

Time taken: $O(N)$

Total Time: $O(N) + O(N) + O(N) = O(3N)$, which is $O(n)$.

- **Solve 2 (using iteration method):**

We can represent the recursion with the following relation:

- $T(n) = T(n-1) + c$
- Substitute $T(n-1)$: $T(n) = (T(n-2) + c) + c = T(n-2) + 2c$
- Substitute $T(n-2)$: $T(n) = (T(n-3) + c) + 2c = T(n-3) + 3c$
- If we continue this pattern for k steps, we get the general form:
 $T(n) = T(n-k) + kc$
- The base case is reached when $n - k = 0$, meaning $k = n$.
- Substituting n for k : $T(n) = T(n-n) + nc = T(0) + nc$
- Since $T(0)$ takes a constant amount of time, we drop the constants, concluding that $T(n) = \Theta(n)$.

6.1.2 Space : $O(N)$

Note from Lecture: Running time is affected by a number of factors other than algorithm efficiency

So, Instead of timing an algorithm, count the number of instructions that it performs

- **Step 1 (The Boolean Array):**

```
vector<bool> pres(n + 2, false);
```

Creating a brand new list in memory directly tied to input size $(N+2)$.

Space used: $O(N)$

- Step 2 (The Recursion Stack):

```
return find_miss(cur + 1, pres, max_val);
```

Every time a function calls itself recursively, the computer has to pause the current function, save its place in memory, and open a new function. This creates a "Call Stack."

In the worst-case scenario (like the 1, 2, 3, 4, 5 example), the function calls itself $N + 1$ times before hitting the base case. The computer has to keep all $N + 1$ function calls open in memory at the exact same time.

Space used: $O(N)$

Total Space Complexity: $O(N)$

$O(N)$ (Array) + $O(N)$ (Stack) = $O(2N)$ → Final Space is $O(N)$.



6.2 Non-Recursive:

6.2.1 Time : $O(n)$

Step 1 (Sorting): We have a for loop from 0 to $n-1$ with a while loop performs a manual swap

to put each number in its correct place. Even though it's a loop inside a loop, each number is

moved to its correct position only once. Therefore, the total number of operations is

proportional to n . So, this step takes $O(n)$.

Step 2 (Scanning): We use a simple for loop to check the numbers one by one, which also

takes $O(n)$.

Total Time: $O(n)+O(n)=O(2n)$, which is $O(n)$.

6.2.2 Space : $O(1)$

The algorithm is In-place. We rearrange the numbers inside the same array A.

We do not use any extra arrays or HashSets.

We only use a few simple variables like n , i and $temp$, so the extra space is constant.

Total Space: $O(1)$.



1. Comparison

<i>Criteria</i>	<i>Non-Recursive</i>	<i>Recursive</i>
<i>Time complexity</i>	$O(n)$	$O(n)$
<i>Space complexity</i>	$O(1)$	$O(n)$
<i>Memory Usage</i>	<i>Very Low (Uses original array)</i>	<i>High (Uses extra vector & stack)</i>
<i>Implementation</i>	<i>Needs a bit of logic (Swapping)</i>	<i>Easier to think of (Intuitive)</i>
<i>Scalability</i>	<i>Highly Scalable</i>	<i>Limited by Stack Size (May Crash)</i>
<i>Approach</i>	<i>In-place Sorting: Uses the array itself as a map by moving each number to its</i>	<i>Indirect Marking: Uses an external boolean map to track existing numbers.</i>

	<i>index.</i>	
<i>Readability</i>	<i>Moderate: Requires understanding of the Cyclic Sort swap logic (more "Engine-like").</i>	<i>High: The code is clean, intuitive, and follows a "Human-like" logic.</i>
<i>Risks</i>	<i>Data Mutation: It modifies the original input array (might be a risk if data is needed later).</i>	<i>Stack Overflow: Risk of crashing if N is very large (e.g., 10^6) due to stack depth.</i>

7a) Performance Insight:

The performance of an algorithm is not just about how fast it runs, but how it manages system resources (Time & Memory).

- **Time Efficiency:** Both algorithms are efficient with a Linear Time Complexity $O(n)$. This means they can handle large datasets without a massive drop in speed.
- **Memory Efficiency:** This is where they differ significantly. The Recursive approach is "Memory-Expensive" because it requires extra space for the boolean array and the call stack. On the other hand,

the Non-Recursive (Cyclic Sort) is "Memory-Efficient" as it works In-place, using only the existing array.

- **Safety:** The Non-Recursive approach is more stable for "Big Data" because it avoids the risk of a Stack Overflow, which is a common failure point for deep recursion.

7b) When to use each?

Choosing the right algorithm depends on the specific requirements of your project:

Use the Recursive Approach when:

- **Simplicity is Priority:** When the code needs to be written quickly and must be very easy for other developers to read and maintain.
- **Small Data:** When you know the input size N is small, so the memory usage of the stack won't be an issue.
- **Prototyping:** Ideal for academic assignments or quick proofs of concept where clean logic is more important than hardware optimization.

Use the Non-Recursive (Cyclic Sort) when:

- **Memory is Limited:** Essential for embedded systems, mobile apps, or any environment with restricted RAM.
- **Massive Datasets:** When dealing with millions of elements where recursion would definitely crash the system.
- **Performance-Critical Systems:** When you need the **Optimal** solution ($O(n)$ Time and $O(1)$ Space) to ensure the highest possible efficiency.



1. Another Alternatives

8.1 Rejected Alternatives for the Iterative Solution:

8.1.1 Standard Sorting ($O(N \log N)$ Time, $O(1)$ Space)

Approach: Sort the array using `std::sort`, then linearly scan to find the first gap.

Reason for Rejection: The problem explicitly requires an efficient $O(N)$ time complexity algorithm. Sorting takes $O(N \log N)$, which is mathematically slower and sub-optimal for this specific problem.

8.1.2 Hash Set / External Boolean Array ($O(N)$ Time, $O(N)$ Space)

Approach: Use a secondary data structure to mark the presence of elements.

Reason for Rejection: It violates the optimization goal. The assignment implicitly favors an auxiliary space approach (like Cyclic Sort) to achieve $O(1)$ constant space. Using a Hash Set consumes unnecessary memory resources.

8.1.3 Brute Force ($O(N^2)$ Time, $O(1)$ Space)

Approach: For every integer starting from 1, scan the entire array to check if it exists.

Reason for Rejection: Extremely inefficient. For an input of 10^5 elements, this would result in 10^{10} operations, inevitably causing a Time Limit Exceeded (TLE) error.

8.2 Rejected Alternatives for the Recursive Solution:

8.2.1 Divide and Conquer / Partitioning ($O(N)$ Time)

Approach: Use a Quick-Sort-like partitioning to move negatives and out-of-bound numbers, then recursively search the valid half.

Reason for Rejection: Over-engineering. This approach is notoriously complex to implement for this specific problem. Our Linear Recursion with a frequency array achieves the same $O(N)$ goal with significantly cleaner and more readable code.

8.2.2 Recursion with Pass-by-Value (Missing the &)

Approach: Passing the vector to the recursive function without the reference operator (&).

Reason for Rejection: Critical Memory Overhead. Passing by value creates a full copy of the array for every recursive call. If the recursion depth is N , this results in $O(N^2)$ time and $O(N^2)$ space, guaranteeing a **Memory Limit Exceeded (MLE)** or **Stack Overflow** crash.

8.2.3 In-place Recursive Swapping

Approach: Implementing the Cyclic Sort swapping logic recursively.

Reason for Rejection: High risk of infinite loops (especially with duplicate values) and extremely difficult to debug. Separating the recursion to just "Read-Only" logic is much safer and maintains code purity.